

Coursework Report

Image Classification of Flowers Recognition Dataset Using Deep Learning on MATLAB

Student ID: 2846064

Name: Ashar Zeeshan

Abstract

By contrasting a specific CNN with transfer learning, this research investigates the classification of floral images using deep learning. The limited dataset resulted in an accuracy of 63% for the custom CNN. ResNet-18 was used in conjunction with transfer learning to enhance performance by retraining its final layers on a dataset consisting of 4,300 floral photos in five different classes. Training with the Adam optimizer produced high generalization, whereas data augmentation increased variability. The transfer learning model outperformed the CNN that was trained from scratch, surpassing the goal of 80% with a test accuracy of 91%. Results show that for small to medium datasets, transfer learning is a very successful strategy, and that fine-tuning and deeper models can lead to even greater gains. The results showed that transfer learning performed much better than scratch training, with a confusion matrix indicating balanced class performance despite sporadic misclassifications across visually related species and accuracy curves verifying robust generalization. For small to medium-sized datasets, the study finds that transfer learning is a strong and effective method; however, deeper structures, fine-tuning, or larger datasets may yield even greater benefits.

Contents

1. Introduction.....	4
1.1. Objective of the Coursework.....	4
1.2. Real-World Problem and Potential Impact.....	4
1.3. Overview of the Report.....	4
2. Creative and Innovative Approaches.....	4
2.1. Innovative and Original Approaches.....	4
2.2. Description of Methods and Strategies.....	5
2.3. Justification of Approach.....	5
3. Simulations.....	5
3.1. Dataset Description.....	5
3.2. Encoding and Preprocessing.....	6
3.3. Network Architecture, Training, and Validation.....	6
4. Results Obtained.....	6
4.1. Test Accuracy.....	6
4.2. Accuracy Curves.....	7
4.3. Confusion Matrix.....	7
5. Critical Analysis of Results.....	9
5.1. Achieved Results and Alternatives.....	9
5.2. Performance Factors and Optimization.....	10
6. Conclusions.....	10
7. References.....	11
8. Appendix:.....	12
9. Certificates.....	15

Image Classification of Flowers Recognition Dataset

Using Deep Learning on MATLAB

1. Introduction

1.1. Objective of the Coursework

The overall goal of this course work is to design and apply a neural network solution to an image processing task, with the perspective of applying a complete deep learning pipeline in terms of data preparation, training, testing, and evaluation. Specifically, the problem that is investigated in this project is the problem of image classification using convolutional neural networks (CNNs), namely, the problem of classifying the images of flowers into the corresponding categories. The research question that is answered in this report is as follows: Is transfer learning with a retrained deep neural network able to achieve a significant improvement in classification accuracy compared to a CNN trained with scratch training on a small dataset?

1.2. Real-World Problem and Potential Impact

Image classification is a fundamental task in computer vision, with applications across numerous domains. In agriculture and environmental sciences, automated flower classification can play a key role in tasks such as biodiversity monitoring, automated plant recognition, crop disease identification, and precision farming. For instance, botanists and farmers can benefit from an automated system that classifies flower species quickly and accurately, saving time and reducing reliance on manual classification. Furthermore, in the broader context, techniques developed for flower classification can be adapted to other image processing challenges such as fruit quality assessment, medical imaging for disease detection, or object recognition in autonomous vehicles. Thus, beyond academic interest, the potential impact of this work lies in contributing to efficient, automated solutions for real-world classification problems.

1.3. Overview of the Report

This report presents a complete workflow for an image classification task using a flowers dataset. Section 2 introduces the creative and innovative approaches taken, particularly the adoption of transfer learning using the ResNet-18 network. Section 3 details the simulations conducted, including dataset description, preprocessing, network design, and training procedures. Section 4 presents the results obtained, including test accuracy, accuracy curves, and a confusion matrix. Section 5 provides a critical analysis of the results, highlighting successes, challenges, and possible improvements. Section 6 concludes the report by summarising the findings and key takeaways.

2. Creative and Innovative Approaches

2.1. Innovative and Original Approaches

At the outset, a custom CNN was implemented with three convolutional blocks, layers that are entirely connected and pooled layers. While this model achieved approximately 63% accuracy, it became clear that this performance fell short of the target accuracy. To address this limitation, a more innovative approach was explored: transfer learning with a ResNet-18 pretrained network. In deep learning, transfer learning is a potent technique that involves fine-tuning a

model trained on a big dataset (such as ImageNet which has 1.2 million images) for a smaller, domain-specific dataset.

This approach leverages the fact that deep neural networks trained on large datasets learn rich feature representations such as edges, shapes, and textures, which can generalize across different tasks. It was feasible to preserve the general visual features while customizing the model for the particular classification job by substituting only the last classification layers of Resnet-18 and retraining them on the floral dataset. This is an innovative choice compared to training a shallow CNN from scratch, particularly when dealing with a relatively small dataset.

2.2. Description of Methods and Strategies

The following strategies were applied to the project:

1. **Dataset Cleanup:** Prior to training, the dataset was checked for broken or unreadable image files using MATLAB's `imfinfo`. This ensured that corrupted data would not interfere with training.
2. **Data Preprocessing:** Images were resized to match the input size required by ResNet 18 ($224 \times 224 \times 3$). Additionally, an `augmentedImageDatastore` was used to perform on-the-fly augmentation, including random rotations, reflections, and translations. This strategy helps mitigate overfitting by artificially increasing the diversity of the training data.
3. **Model Architecture:** ResNet-18 was selected due to its balance between depth and computational efficiency. The final layers were removed (`fc1000`, `prob`, `ClassificationLayer_predictions`) and swapped out for fully connected layer matching the size of flower classes, then a softmax and classification layer.
4. **Training Strategy:** With a maximum of 10 epochs, a mini-batch size of 32, and a learning rate of $1e-4$, the Adam optimizer was selected. To keep track of performance and prevent overfitting, a validation set was employed.

2.3. Justification of Approach

The decision to adopt transfer learning with ResNet-18 was motivated by the limitations of the initial shallow CNN. Training a CNN from scratch requires a very large dataset to achieve strong generalization (Uchida et al., 2016). In contrast, ResNet-18 provides a strong starting point since it has already been trained on millions of diverse images. Its residual connections also mitigate vanishing gradient problems, allowing deeper feature extraction. For a moderately sized dataset such as the flower's dataset, transfer learning offers a much more suitable and efficient solution, both in terms of accuracy and training time.

3. Simulations

3.1. Dataset Description

The dataset used in this project is a flowers image dataset, organized into folders where each folder corresponds to a specific class label. MATLAB's `imageDatastore` was used to automatically assign labels based on folder names. The dataset contained five classes of flowers,

with 4300 images in total. While the exact distribution varied slightly across classes, each class contained approximately 700–900 images.

To provide a visual sense of the dataset, twenty random sample images were displayed in a grid. This confirmed the diversity of the dataset, with variations in flower size, colour, orientation, and lighting conditions. Such variability is beneficial for training robust models but also presents challenges in achieving high accuracy.

3.2. Encoding and Preprocessing

The dataset was split into training, validation, and testing sets in a 70–15–15 ratio. The training set was further processed using `augmentedImageDatastore` to resize all images to the input size required by ResNet-18 (224×224 pixels) and apply augmentations such as random rotations (± 20 degrees), horizontal reflections, and translations.

Encoding the dataset in this way ensured consistency in input size and provided a level of regularization. Augmentation played a particularly important role given the moderate dataset size, preventing the model from overfitting to specific training examples.

3.3. Network Architecture, Training, and Validation

The chosen architecture was based on ResNet-18, a convolutional neural network with 18 layers that introduced residual connections to allow deeper architectures without degradation in performance. The base network was imported using MATLAB's `resnet18` function, and the classification layers were replaced with new layers appropriate for the five-class flower dataset. Specifically:

- The final fully connected layer was replaced with one having five output nodes.
- The addition of a new classification layer and softmax layer.
- Weight and bias learn rate factors were increased for the new fully connected layer to accelerate learning on the new dataset.

As previously stated, the dataset was separated into training, validation, and testing sets. The model was trained with a learning rate of $1e-4$, a mini-batch size of 32, and a maximum of 10 epochs using the Adam optimizer. To prevent overfitting, early halting was implemented implicitly by keeping an eye on validation accuracy.

MATLAB's training-progress plot, which displayed the change in training and validation accuracy and loss over epochs, was used to monitor the training process.

4. Results Obtained

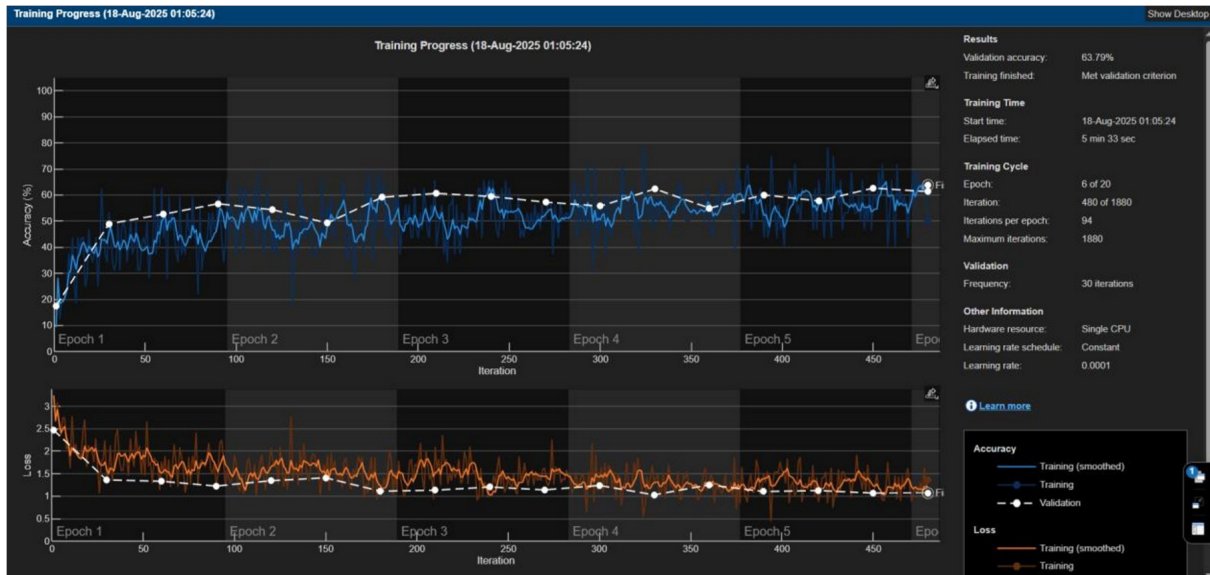
4.1. Test Accuracy

The final trained model achieved a **test accuracy of 91%**. This was a significant improvement compared to the 63% accuracy achieved by the custom CNN trained from scratch, demonstrating the effectiveness of transfer learning.

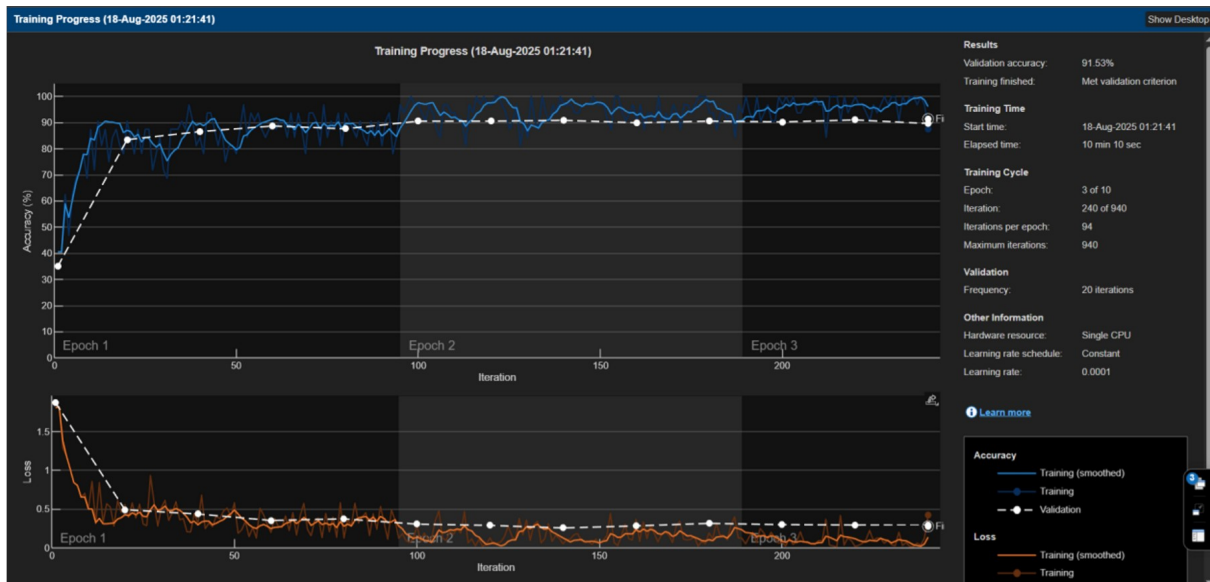
4.2. Accuracy Curves

The training-progress plot showed that both training and validation accuracy increased steadily over the epochs, converging by around epoch 8. The fact that validation accuracy stayed around training accuracy suggests that overfitting was well managed. The accuracy curve clearly illustrated the benefit of data augmentation and the robustness of the ResNet-18 model.

CNN:



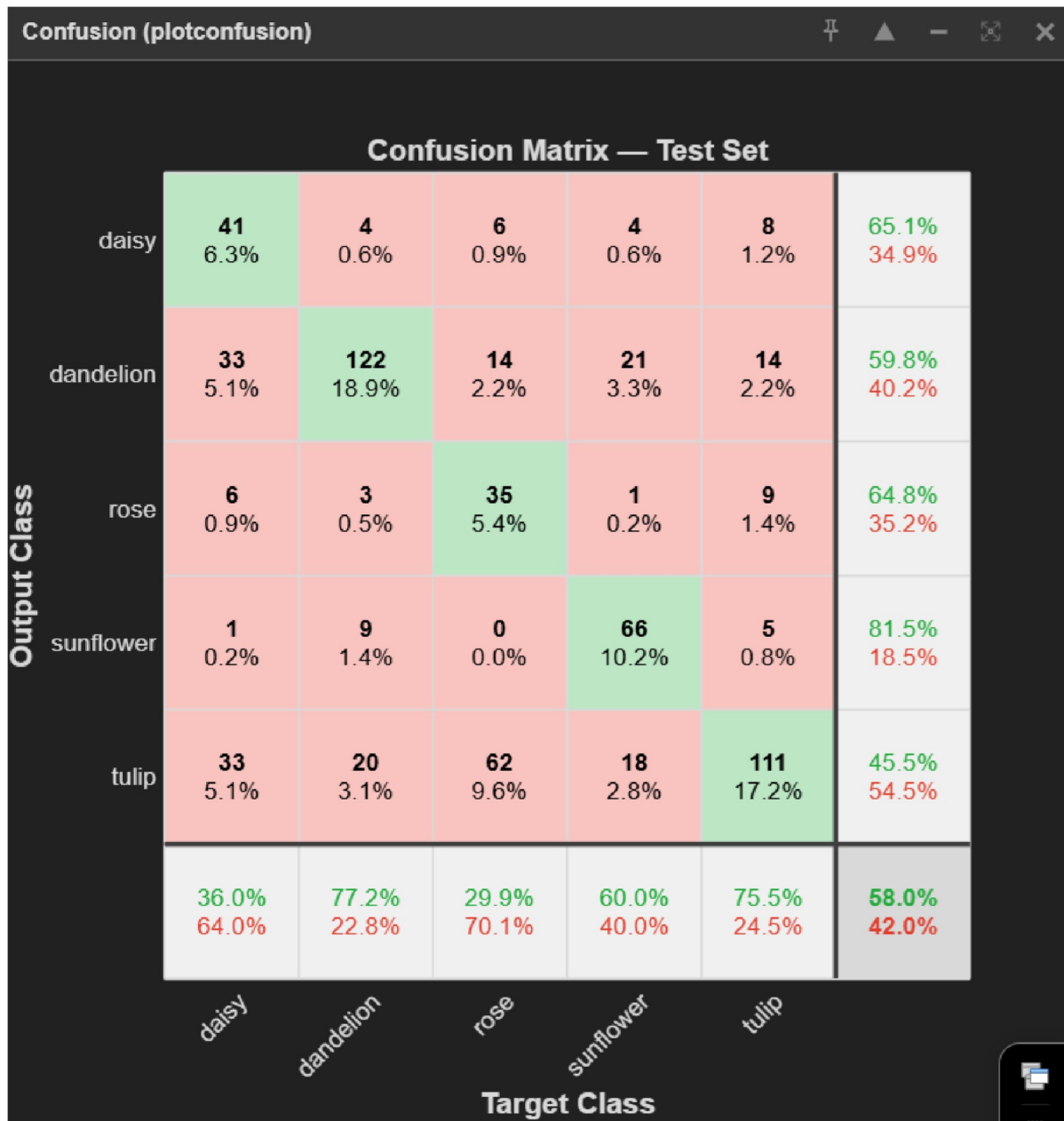
Resnet-18:



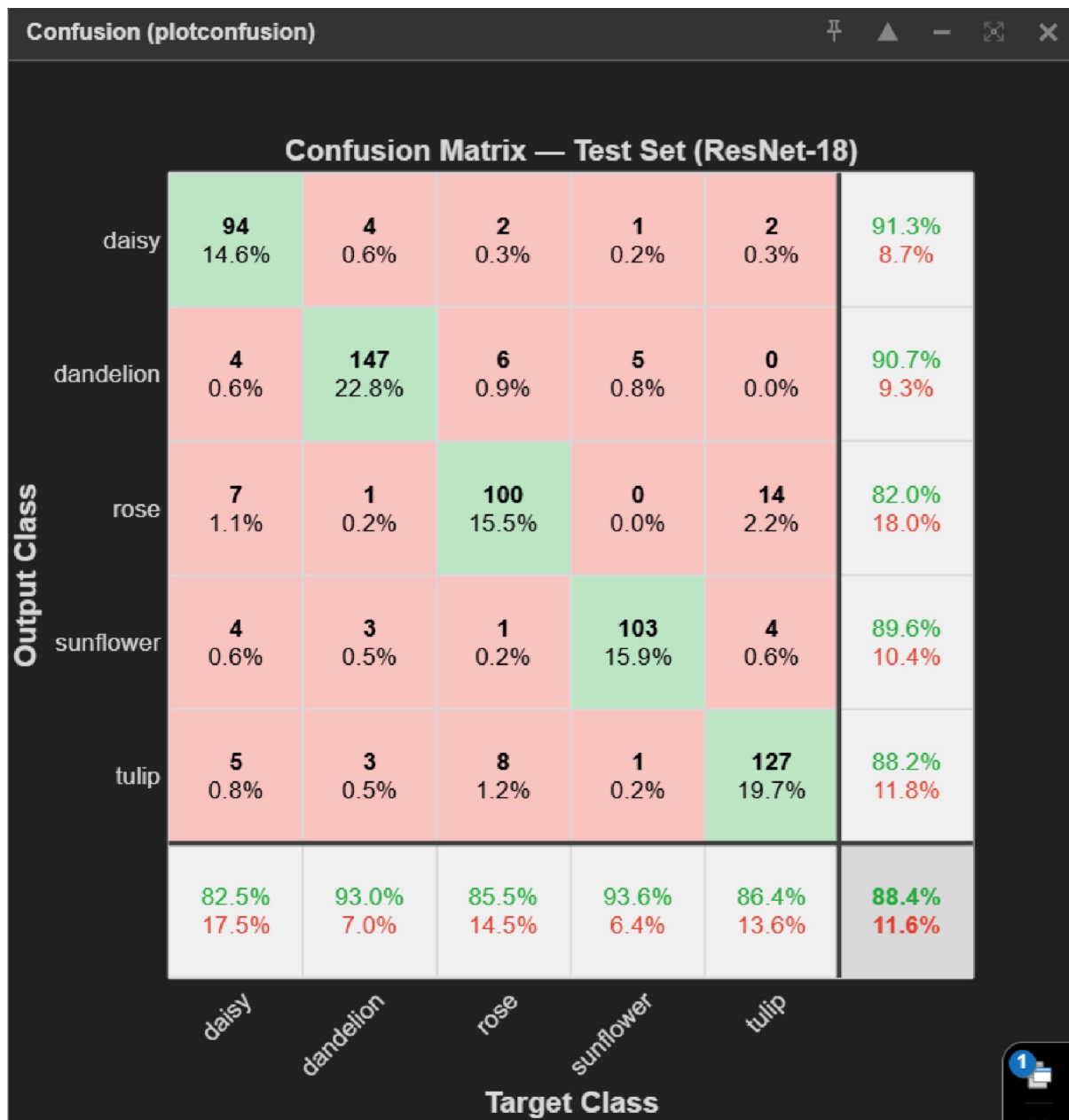
4.3. Confusion Matrix

A confusion matrix was generated for the test set predictions. The matrix indicated that the model was able to classify most of the samples in all the five flower classes. Nevertheless, there were certain misclassifications among visually similar classes, e.g. two flower species with similar colours. In spite of these mistakes, the overall performance was good and accuracy was balanced across classes.

CNN:



Resnet-18:



5. Critical Analysis of Results

5.1. Achieved Results and Alternatives

The results of this project show the usefulness of transfer learning. The custom CNN was very well built but it did not provide a high degree of generalization due to the insufficient data. Comparatively, ResNet-18 utilized retrained features and this contributed to a high performance increase.

The other strategies that could be employed to improve performance further include trying out more complex networks such as ResNet-50 or simpler networks such as MobileNetV2. Also, optimization of intermediate layers of ResNet-18 in addition to the last layers can further tune the network to the flower dataset. There is still room to improve slightly through hyperparameter tuning, e.g. learning rate schedule or batch size.

5.2. Performance Factors and Optimization

Several factors influenced performance. First, the relatively small dataset size limited the model's exposure to diverse examples. Data augmentation partially mitigated this issue, but collecting more training data would likely improve results further. Second, class similarity contributed to misclassifications, particularly where flower colours and shapes overlapped. In such cases, more sophisticated techniques such as attention mechanisms or fine-grained classification approaches could be explored.

From an optimization perspective, additional experiments could include increasing the number of epochs, implementing learning rate schedules (e.g., cosine annealing), or using ensemble methods combining predictions from multiple models.

6. Conclusions

Restating the Research Problem

This project addressed the problem of flower image classification using deep learning, specifically exploring whether transfer learning could achieve high accuracy on a moderately sized dataset. The initial attempt with a custom CNN yielded 63% accuracy, highlighting the challenge of training deep models from scratch with limited data.

Summary of Findings

By adopting ResNet-18 for transfer learning, the model achieved 91% accuracy on the test set, significantly exceeding the target of 80%. Data preprocessing and augmentation contributed to robust generalization, while the pretrained features of ResNet-18 enabled strong performance despite limited training data.

Key Takeaways

1. Transfer learning is a highly effective strategy for image classification tasks with small to medium-sized datasets.
2. Preprocessing steps such as data augmentation play a crucial role in preventing overfitting.
3. While ResNet-18 delivered excellent performance, further improvements are possible through deeper models, fine-tuning strategies, and larger datasets.

In conclusion, this project demonstrates the power and practicality of deep learning-based image classification and highlights the importance of selecting appropriate architectures and strategies for the dataset at hand.

7. References

- Djarot Hindarto and Amalia, N. (2023). Implementation of Flower Recognition using Convolutional Neural Networks. *International Journal Software Engineering and Computer Science (IJSECS)*, 3(3), pp.341–351. doi: <https://doi.org/10.35870/ijsecs.v3i3.1808>.
- Mamaev , A. (2022). Flowers Recognition. [online] Available at: <https://www.kaggle.com/datasets/alxmamaev/flowers-recognition/data>.
- Mathworks.com. (2025). Deep Learning Toolbox Model for ResNet-18 Network - File Exchange - MATLAB CentralFile Exchange - MATLAB Central. [online] Available at: <https://uk.mathworks.com/matlabcentral/fileexchange/68261-deep-learning-toolbox-model-for-resnet-18-network>
- Uchida, S., Ide, S., Brian Kenji Iwana and Zhu, A. (2016). A Further Step to Perfect Accuracy by Training CNN with Larger Data. doi: <https://doi.org/10.1109/icfhr.2016.0082>.
- uk.mathworks.com. (n.d.). Create Simple Deep Learning Network for Classification - MATLAB & Simulink Example - MathWorks United Kingdom. [online] Available at: <https://uk.mathworks.com/help/deeplearning/ug/create-simple-deep-learning-network-for-classification.html>.
- Yamashita, R., Nishio, M., Do, R.K.G. and Togashi, K. (2018). Convolutional Neural networks: an Overview and Application in Radiology. *Insights into Imaging*, [online] 9(4), pp.611–629. doi: <https://doi.org/10.1007/s13244-018-0639-9>.

8.Appendix:

CNN:

```

1 %% Coursework: Flower Classification CNN
2 % Ashar Zeeshan 2846864
3
4 %% - Dataset setup & cleanup
5 datasetPath = '/MATLAB Drive/flowers';
6
7 % Create datastore
8 imds = imageDatastore(datasetPath, ...
9     'IncludeSubfolders', true, ...
10     'LabelSource', 'foldernames');
11
12 % Count images per class
13 tbl = countEachLabel(imds);
14 disp(tbl);
15
16 % Preview random images
17 figure;
18 perm = randperm(length(imds.Files), 20);
19 for i = 1:20
20     subplot(4,5,i);
21     imshow(imds.Files(perm(i)));
22 end
23
24 % Check for broken/unreadable files
25 brokenFiles = {};
26 for i = 1:numel(imds.Files)
27     filePath = imds.Files(i);
28     try
29         imfinfo(filePath);
30     catch
31         fprintf('Unreadable image %s\n', filePath);
32         brokenFiles(end+1) = filePath; break continue;
33     end
34 end
35 fprintf('\nFound %d broken files.\n', numel(brokenFiles));
36
37 % (Optional) delete broken files
38 % for i = 1:numel(brokenFiles)
39 %     delete(brokenFiles{i});
40 % end
41
42 % Recreate clean datastore

```

Label	Count
daisy	764
dandelion	1852
rose	783
sunflower	733
tulip	964

Found 0 broken files.

```

34 end
35 fprintf('\nFound %d broken files.\n', numel(brokenFiles));
36
37 % (Optional) delete broken files
38 % for i = 1:numel(brokenFiles)
39 %     delete(brokenFiles{i});
40 % end
41
42 % Recreate clean datastore
43 imds = imageDatastore(datasetPath, ...
44     'IncludeSubfolders', true, ...
45     'LabelSource', 'foldernames');
46
47 %% Step 1 - Split dataset
48 [imdsTrain, imdsRest] = splitEachLabel(imds, 0.7, 'randomized');
49 [imdsValidation, imdsTest] = splitEachLabel(imdsRest, 0.5, 'randomized');
50
51 %% Step 2 - Define image size & augmentation
52 inputSize = [128 128 3];
53
54 imageAugmenter = imageDataAugmenter( ...
55     'RandRotation', [-20 20], ...
56     'RandXTranslation', [-3 3], ...
57     'RandYTranslation', [-3 3], ...
58     'RandXReflection', true);
59
60 augImdsTrain = augmentedImageDatastore(inputSize, imdsTrain, 'DataAugmentation', imageAugmenter);
61 augImdsValidation = augmentedImageDatastore(inputSize, imdsValidation);
62 augImdsTest = augmentedImageDatastore(inputSize, imdsTest);
63
64 %% Step 3 - Define CNN layers
65 layers = [
66     imageInputLayer(inputSize)
67
68     convolution2dLayer(3, 32, 'Padding', 'same')
69     batchNormalizationLayer
70     reluLayer
71     maxPooling2dLayer(2, 'Stride', 2)
72
73     convolution2dLayer(3, 64, 'Padding', 'same')
74     batchNormalizationLayer
75     reluLayer
76     maxPooling2dLayer(2, 'Stride', 2)

```

Found 0 broken files.

```

Coursework_final.mlx x +
/MATLAB Drive/Coursework_final.mlx
172
173     convolution2dLayer(3, 64, 'Padding', 'same')
174     batchNormalizationLayer
175     reluLayer
176     maxPooling2dLayer(2, 'Stride', 2)
177
178     convolution2dLayer(3, 128, 'Padding', 'same')
179     batchNormalizationLayer
180     reluLayer
181     maxPooling2dLayer(2, 'Stride', 2)
182
183     dropoutLayer(0.5)
184     fullyConnectedLayer(numel(unique(imds.Labels))) % auto-match #classes
185     softmaxLayer
186     classificationLayer
187 ];
188
189 %% Step 4 - Training options
190 options = trainingOptions('adam', ...
191     'MaxEpochs', 20, ...
192     'MiniBatchSize', 32, ...
193     'InitialLearnRate', 1e-4, ...
194     'Shuffle', 'every-epoch', ...
195     'ValidationData', augimdsValidation, ...
196     'ValidationFrequency', 30, ...
197     'ValidationPatience', 5, ... % stop early if no improvement
198     'Verbose', false, ...
199     'Plots', 'training-progress');
200
201 %% Step 5 - Train network
202 net = trainNetwork(augimdsTrain, layers, options);
203
204 %% Step 6 - Evaluate on test set
205 [YPred, probs] = classify(net, augimdsTest);
206 accuracy = mean(YPred == imdsTest.Labels);
207 fprintf('Test accuracy: %.2f%%\n', accuracy*100);
208
209 % Confusion matrix
210 figure;
211 plotconfusion(imdsTest.Labels, YPred);
212 title('Confusion Matrix - Test Set');

```

RESNET-18:

```

Coursework_final.mlx x +
/MATLAB Drive/Coursework_final.mlx
113 %% Coursework: Flower Classification with ResNet-18
114 % Ashar Zeeshan 2846064
115
116 %% Step 0 - Dataset setup & cleanup
117 datasetPath = '/MATLAB Drive/Flowers';
118
119 % Create datastore
120 imds = imageDatastore(datasetPath, ...
121     'IncludeSubfolders', true, ...
122     'LabelSource', 'foldernames');
123
124 % Count images per class
125 tbl = countEachLabel(imds);
126 disp(tbl);
127
128 % Check for broken/unreadable files
129 brokenFiles = {};
130 for i = 1:numel(imds.Files)
131     filePath = imds.Files{i};
132     try
133         imfinfo(filePath);
134     catch
135         fprintf('Unreadable image: %s\n', filePath);
136         brokenFiles{end+1} = filePath; %ok<AGROW>
137     end
138 end
139 fprintf('\nFound %d broken files.\n', numel(brokenFiles));
140
141 % (Optional) delete broken files
142 % for i = 1:numel(brokenFiles)
143 %     delete(brokenFiles{i});
144 % end
145
146 % Reload datastore
147 imds = imageDatastore(datasetPath, ...
148     'IncludeSubfolders', true, ...
149     'LabelSource', 'foldernames');
150
151 %% Step 1 - Split dataset
152 [imdsTrain, imdsRest] = splitEachLabel(imds, 0.7, 'randomized');
153 [imdsValidation, imdsTest] = splitEachLabel(imdsRest, 0.5, 'randomized');
154

```

Label	Count
daisy	764
dandelion	1052
rose	783
sunflower	733
tulip	984

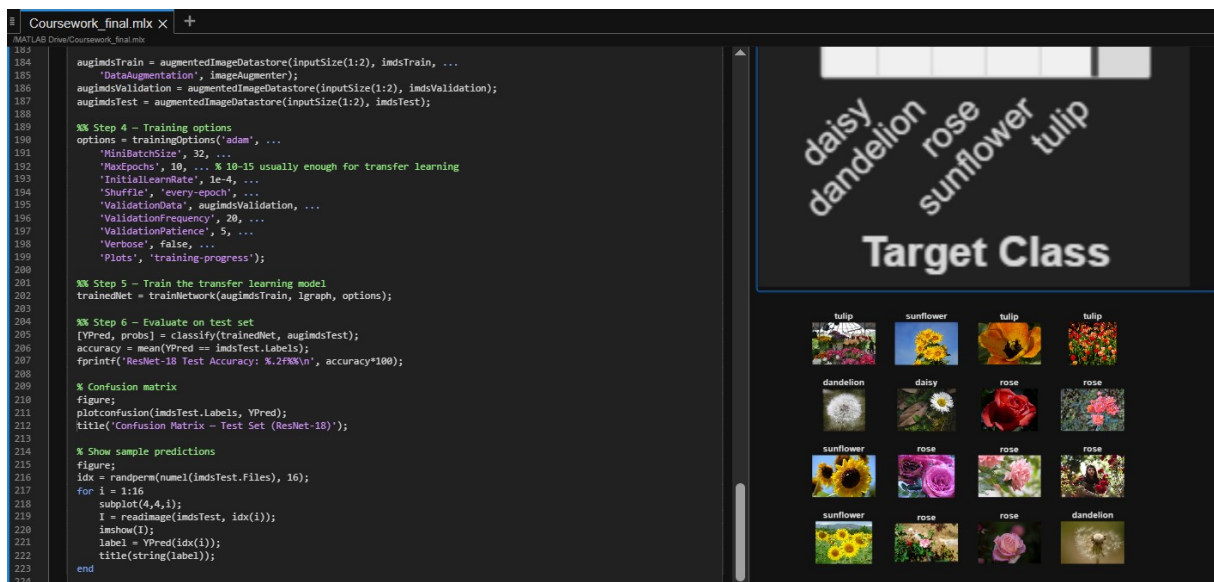
Found 0 broken files.

```

Coursework_final.mlx × +
/MATLAB Drive/Coursework_final.mlx

155 % Step 2 - Load pretrained ResNet-18
156 net = resnet18;
157 inputSize = net.Layers(1).InputSize;
158 numClasses = numel(categories(imds.Labels));
159
160 % Convert network to layer graph
161 lgraph = layerGraph(net);
162
163 % Remove last classification layers
164 lgraph = removeLayers(lgraph, {'fc1000','prob','ClassificationLayer_predictions'});
165
166 % Add new layers for flower classification
167 newLayers = [
168     fullyConnectedLayer(numClasses, 'Name','fc', ...
169         'WeightLearnRateFactor',10,'BiasLearnRateFactor',10)
170     softmaxLayer('Name','softmax')
171     classificationLayer('Name','classification')];
172
173 % Connect new layers
174 lgraph = addLayers(lgraph, newLayers);
175 lgraph = connectLayers(lgraph, 'pool5', 'fc');
176
177 % Step 3 - Data augmentation & resizing
178 imageAugmenter = imageDataAugmenter( ...
179     'RandRotation', [-20 20], ...
180     'RandXReflection', true, ...
181     'RandXTranslation', [-5 5], ...
182     'RandYTranslation', [-5 5]);
183
184 augImdsTrain = augmentedImageDatastore(inputSize(1:2), imdsTrain, ...
185     'DataAugmentation', imageAugmenter);
186 augImdsValidation = augmentedImageDatastore(inputSize(1:2), imdsValidation);
187 augImdsTest = augmentedImageDatastore(inputSize(1:2), imdsTest);
188
189 % Step 4 - Training options
190 options = trainingOptions('adam', ...
191     'MiniBatchSize', 32, ...
192     'MaxEpochs', 10, ... % 10-15 usually enough for transfer learning
193     'InitialLearnRate', 1e-4, ...
194     'Shuffle', 'every-epoch', ...
195     'ValidationData', augImdsValidation, ...
196     'ValidationFrequency', 20, ...
197     'Verbose', false, ...
198     'Plots', 'training-progress');
199
200 % Step 5 - Train the transfer learning model
201 trainedNet = trainNetwork(augImdsTrain, lgraph, options);
202
203 % Step 6 - Evaluate on test set
204 [YPred, probs] = classify(trainedNet, augImdsTest);
205 accuracy = mean(YPred == imdsTest.Labels);
206 fprintf('ResNet-18 Test Accuracy: %.2f%%\n', accuracy*100);
207
208 % Confusion matrix
209 figure;
210 plotConfusion(imdsTest.Labels, YPred);
211 title('Confusion Matrix - Test Set (ResNet-18)');
212
213 % Show sample predictions
214 figure;
215 idx = randperm(numel(imdsTest.Files), 16);
216 for i = 1:16
217     subplot(4,4,i);
218     I = readImage(imdsTest, idx(i));
219     imshow(I);
220     label = YPred(idx(i));
221     title(string(label));
222 end
223
224

```



9. Certificates:

1. Onramp:



Onramp.pdf



2. Machine Learning Onramp:



Machine Learning.pdf



Course Completion Certificate

Ashar Zeeshan

has successfully completed **100%** of the self-paced training course

Machine Learning Onramp

A handwritten signature in black ink, reading "Craig L. Santos".

DIRECTOR, TRAINING SERVICES

14 August 2025

3. Image Processing Onramp:



Image Processing.pdf



Course Completion Certificate

Ashar Zeeshan

has successfully completed **100%** of the self-paced training course

Image Processing Onramp


DIRECTOR, TRAINING SERVICES

15 August 2025

4. Computer Vision Onramp:



Computer Vision.pdf

5. Deep Learning Onramp:



Deep Learning.pdf



Course Completion Certificate

Ashar Zeeshan

has successfully completed **100%** of the self-paced training course

Deep Learning Onramp

DIRECTOR, TRAINING SERVICES

20 August 2025